

BIGINT CLASS

Software Requirements Specification (SRS)

Trainee Implementation Guide

Version 1.0 | C++ Operator Overloading Training Module

Prepared for: Software Engineering Trainees
Language: C++ (Operator Overloading & String Arithmetic)

INTERNAL TRAINING DOCUMENT — NOT FOR PUBLIC DISTRIBUTION

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional requirements for implementing a BigInt class in C++. It is intended to guide trainees through completing a partially-written codebase by specifying exactly what each method and operator must do.

By the end of this exercise, trainees will demonstrate mastery of:

- Operator overloading in C++ (arithmetic, relational, stream, increment/decrement)
- String-based big number arithmetic (addition, subtraction, multiplication, division, modulus)
- Sign handling for negative numbers
- Edge-case handling: division by zero, leading zeros, zero inputs
- Class design with private helpers and public interface

1.2 Project Overview

The BigInt class represents arbitrarily large integers that exceed the range of built-in C++ integer types (int, long, long long). Numbers are stored internally as strings and all arithmetic is performed digit-by-digit. The skeleton code (class declarations with empty method bodies and a commented-out test suite) is provided. Trainees must implement all method bodies.

1.3 Scope

The BigInt class covers the following operations:

- Construction from integer literals, strings, and copy
- All standard arithmetic: +, -, *, /, %
- All relational comparisons: ==, !=, <, <=, >, >=
- Pre/post increment and decrement: ++x, x++, --x, x--
- Unary negation and plus: -x, +x
- Stream I/O: cin >> x, cout << x

1.4 Definitions and Acronyms

Term	Definition
BigInt	An arbitrarily large integer stored as a digit string
Magnitude	The absolute value of a number, ignoring its sign
DXA	Document measurement unit used internally by .docx
Trainee	A junior developer completing this exercise
Signed arithmetic	Arithmetic that correctly handles both positive and negative operands
Leading zeros	Zeros at the front of a digit string that carry no value (e.g. "007" → "7")

2. Internal Representation

The BigInt class stores its value as two private member variables:

Variable	Type	Description
number	string	The digits of the integer, stored without a leading minus sign. Example: "-42" is stored as "42" with isNegative = true
isNegative	bool	True if the number is negative, false otherwise. Zero is always stored as positive (isNegative = false)

Two private helper functions support all operations:

2.1 removeLeadingZeros()

Removes any leading zero characters from the number string. After every arithmetic operation, call this function to normalize the result.

- Scan the string from left to right and erase leading '0' characters
- If the result is empty (all digits were zero), set number = "0"
- If number becomes "0", ensure isNegative = false (negative zero is not valid)

```
// Examples:  
"007" → "7"  
"000" → "0" (and isNegative → false)  
"00123" → "123"
```

2.2 compareMagnitude(const BigInt& other) const

Compares the absolute values of two BigInts without considering their signs. Returns:

Return Value	Meaning
1	this > other
0	this == other
-1	this < other

Implementation strategy:

- First compare string lengths: a longer string always represents a larger magnitude
- If lengths are equal, compare the strings lexicographically (since both contain only digit characters, standard string comparison works correctly)

3. Constructors

3.1 Default Constructor — BigInt()

Initializes the BigInt to represent the value zero.

- Set number = "0"
- Set isNegative = false

3.2 Integer Constructor — BigInt(int64_t value)

Initializes the BigInt from a 64-bit signed integer.

- If value < 0: set isNegative = true and convert the absolute value to a string
- If value >= 0: set isNegative = false and convert value to a string
- Use std::to_string() for the conversion

```
BigInt a(12345);    // number = "12345", isNegative = false
BigInt b(-67890);   // number = "67890", isNegative = true
BigInt c(0);        // number = "0", isNegative = false
```

3.3 String Constructor — BigInt(const string& str)

Initializes the BigInt from a string representation of an integer.

- If the string starts with '-': set isNegative = true and store the remaining characters as number
- Otherwise: set isNegative = false and store the entire string as number
- Call removeLeadingZeros() after parsing

```
BigInt a("-67890"); // number = "67890", isNegative = true
BigInt b("007");    // number = "7", isNegative = false
BigInt c("-0");      // number = "0", isNegative = false
```

3.4 Copy Constructor — BigInt(const BigInt& other)

Creates a new BigInt as an identical copy of another.

- Copy other.number into this->number
- Copy other.isNegative into this->isNegative

3.5 Destructor — ~BigInt()

No dynamic memory allocation is used, so the destructor body can be left empty. It must still exist.

4. Assignment and Stream Operators

4.1 Assignment Operator — operator=

Assigns the value of another BigInt to this one.

- Guard against self-assignment: if (this == &other) return *this;
- Copy other.number and other.isNegative into this object
- Return *this

4.2 Output Stream Operator — operator<<

Allows BigInt to be printed with cout. Prints the number with a leading minus sign if negative.

```
BigInt a(12345);
BigInt b("-67");
cout << a; // prints: 12345
cout << b; // prints: -67
```

- If isNegative is true and number is not "0": print '-' then number
- Otherwise: print number only

4.3 Input Stream Operator — operator>>

Allows BigInt to be read with cin. Reads one whitespace-delimited token and constructs the BigInt from it.

- Read a string token from is
- Construct a temporary BigInt from that string using the string constructor
- Assign it to num
- Return is

```
BigInt x;
cin >> x; // user types "-999" → x.number = "999", x.isNegative = true
```

5. Arithmetic Operators

This section defines the behavior of all arithmetic operations. For the compound assignment versions ($+=$, $-=$, $*=$, $/=$, $\%=$), implement the logic directly. The binary versions ($+$, $-$, $*$, $/$, $\%$) are implemented by passing a copy of lhs, calling the corresponding compound assignment on it, and returning the result.

5.1 Addition — operator+= and operator+

Adds other to this. Sign rules:

this sign	other sign	Action
positive	positive	Add magnitudes, result is positive
negative	negative	Add magnitudes, result is negative
positive	negative	Subtract smaller magnitude from larger; result sign = sign of larger magnitude
negative	positive	Subtract smaller magnitude from larger; result sign = sign of larger magnitude

Adding two magnitudes (digit-by-digit, right to left):

- Align the two digit strings from the right
- Add corresponding digits plus any carry
- If carry remains after processing all digits, prepend '1' to the result

```

  12345
+ 67890
-----
 80235

```

5.2 Subtraction — operator-= and operator-

Subtracts other from this. Sign rules:

this sign	other sign	Action
positive	negative	Equivalent to adding the magnitude of other (delegate to +=)
negative	positive	Equivalent to adding both magnitudes, result is negative
same sign	same sign	Subtract smaller magnitude from larger; result sign = sign of larger magnitude

Subtracting magnitudes (assuming $|this| \geq |other|$, digit-by-digit right to left):

- Subtract corresponding digits, borrowing from the next column when needed
- Call `removeLeadingZeros()` on the result

```

  80235
-  2000
-----
 78235

```

5.3 Multiplication — `operator*= and operator*`

Multiplies this by other.

- If either operand is zero, the result is zero
- The sign of the result is negative if and only if exactly one operand is negative (XOR rule)
- Implement the standard long multiplication algorithm:

```
// Allocate result array of size (len1 + len2)
// For each digit in number1 (right to left, index i):
//   For each digit in number2 (right to left, index j):
//     result[i + j + 1] += digit1 * digit2
// Handle carry propagation across the result array
// Convert result array back to string, skip leading zeros
```

5.4 Division — `operator/=` and `operator/`

Divides this by other (integer division, truncated toward zero).

- If other is zero: throw `std::runtime_error("Division by zero")`
- If `|this| < |other|`: result is zero
- The sign rule is the same XOR rule as multiplication
- Use repeated subtraction or long division to compute the quotient

Recommended approach — binary search on the quotient:

```
// Find the largest q such that q * |other| <= |this|
// Use low = 0, high = |this|, and binary search
// This is O(log n) divisions on digit-length numbers
```

5.5 Modulus — `operator%= and operator%`

Computes the remainder of dividing this by other.

- If other is zero: throw `std::runtime_error("Division by zero")`
- Compute: `remainder = this - (this / other) * other`
- The sign of the result matches the sign of this (the dividend)

```
BigInt(-10) % BigInt(3)    // result: -1
BigInt(10) % BigInt(-3)    // result: 1
BigInt(10) % BigInt(3)     // result: 1
```

6. Unary and Increment/Decrement Operators

6.1 Unary Negation — operator-()

Returns a copy of this BigInt with the sign flipped. Zero remains zero (not negative zero).

```
BigInt a(5);    -a // returns -5
BigInt b(-5);   -b // returns 5
BigInt c(0);    -c // returns 0 (not -0)
```

6.2 Unary Plus — operator+()

Returns an unchanged copy of this BigInt. Simply copy-construct and return.

6.3 Pre-increment — operator++()

Adds 1 to this BigInt and returns the updated value.

- Equivalent to: `*this += BigInt(1);`
- Return `*this`

6.4 Post-increment — operator++(int)

Returns the original value, then adds 1.

- Save a copy of `*this` as `temp`
- Call `++(*this)` to increment
- Return `temp`

6.5 Pre-decrement — operator--()

Subtracts 1 from this BigInt and returns the updated value.

- Equivalent to: `*this -= BigInt(1);`
- Return `*this`

6.6 Post-decrement — operator--(int)

Returns the original value, then subtracts 1.

- Save a copy of `*this` as `temp`
- Call `--(*this)` to decrement
- Return `temp`

7. Comparison Operators

All six comparison operators must be implemented. `operator==` and `operator<` are declared as friend functions and have direct access to the private members. The remaining four are implemented using those two.

7.1 Equality — `operator==`

Two BigInts are equal if and only if both their signs and their digit strings match.

```
if (lhs.isNegative != rhs.isNegative) return false;  
return lhs.number == rhs.number;
```

7.2 Inequality — `operator!=`

Return `!(lhs == rhs)`.

7.3 Less-Than — `operator<`

Cases to handle:

lhs sign	rhs sign	Rule
negative	positive	lhs < rhs is always true
positive	negative	lhs < rhs is always false
both positive	both positive	lhs < rhs iff lhs < rhs (compareMagnitude returns -1)
both negative	both negative	lhs < rhs iff lhs > rhs (larger magnitude → more negative)

7.4 Less-Than-or-Equal — `operator<=`

Return `(lhs < rhs) || (lhs == rhs)`.

7.5 Greater-Than — `operator>`

Return `!(lhs <= rhs)`.

7.6 Greater-Than-or-Equal — `operator>=`

Return `!(lhs < rhs)`.

7.7 `toString()` Helper

Returns the string representation of the BigInt, identical to what `operator<<` prints.

- If `isNegative` and `number != "0"`: return `"-" + number`
- Otherwise: return `number`

8. Expected Test Output

The test suite in main() is commented out. Uncomment it after implementing all methods. The expected console output for each test group is shown below.

8.1 Test 1 — Constructors and Output

```
a (from int): 12345
b (from string): -67890
c (zero): 0
d (copy of a): 12345
```

8.2 Test 2 — Arithmetic Operations

```
a + b = -55545
a - b = 80235
a * b = -838102050
b / a = -5
a % 100 = 45
```

8.3 Test 3 — Relational Operators

```
a == d: 1
a != b: 1
a < b: 0
a > b: 1
c == 0: 1
```

8.4 Test 4 — Unary and Increment/Decrement

```
-a: -12345
++a: 12346
a--: 12346
a after decrement: 12345
```

8.5 Test 5 — Large Number Operations

```
Very large addition: 1111111101111111100
Very large multiplication: 1219326311370217952237463801111263526900
```

8.6 Test 6 — Edge Cases

```
Division by zero correctly threw error: Division by zero
Multiplication by zero: 0
Negative multiplication: -15
Negative division: -3
Negative modulus: -1
```


9. Common Mistakes & Tips

Mistake	Why It's Wrong	Correct Approach
Forgetting to call <code>removeLeadingZeros()</code>	Results like "007" or "000" break all comparisons and output	Call <code>removeLeadingZeros()</code> at the end of every arithmetic operation
Setting <code>isNegative = true</code> when result is zero	Produces "-0" in output, fails equality checks against <code>BigInt(0)</code>	After any operation, if <code>number == "0"</code> , force <code>isNegative = false</code>
Using <code>int</code> for intermediate digit multiplication	<code>int</code> overflow when multiplying digits * carries in large numbers	Use <code>int</code> or ensure values stay within single-digit range (0-9)
Mixing signed and unsigned string indices	Off-by-one errors when iterating right-to-left	Use <code>int i</code> , not <code>size_t</code> , when iterating backwards so <code>i >= 0</code> works
Not handling the case <code> lhs == rhs </code> with different signs	Subtraction produces "0" but with wrong sign	After subtraction, always run <code>removeLeadingZeros()</code> which resets sign on zero
Division returning wrong sign for negative operands	<code>(-10)/3</code> returns 3 instead of -3	Compute sign before division, divide magnitudes, then apply sign
Modulus sign matching divisor instead of dividend	<code>(-10)%3</code> returns 2 instead of -1	Result sign must match dividend (this), not divisor (other)

10. Implementation Checklist

Use this checklist to track your progress. Every item must be completed before submission.

10.1 Helpers and Constructors

#	Item	Completed?
1	removeLeadingZeros() — strips leading zeros, resets sign if result is zero	☐
2	compareMagnitude() — returns 1, 0, or -1 based on absolute value comparison	☐
3	Default constructor — number = "0", isNegative = false	☐
4	int64_t constructor — handles negative, zero, and positive integers	☐
5	String constructor — parses leading '-', calls removeLeadingZeros()	☐
6	Copy constructor — copies both number and isNegative	☐
7	Assignment operator — self-assignment guard, copies fields, returns *this	☐

10.2 Arithmetic Operators

#	Item	Completed?
8	operator+= — handles all four sign combinations	☐
9	operator-= — delegates opposite-sign cases, handles same-sign subtraction	☐
10	operator*= — long multiplication, XOR sign rule, handles zero	☐
11	operator/= — throws on zero divisor, XOR sign rule, integer truncation	☐
12	operator%= — throws on zero divisor, result sign matches dividend	☐
13	Binary +, -, *, /, % — all use the compound-assignment versions	☐

10.3 Unary and Increment/Decrement Operators

#	Item	Completed?
14	operator-() — flips sign, preserves zero as non-negative	☐
15	operator+() — returns unchanged copy	☐
16	Pre-increment operator++() — adds 1, returns *this	☐
17	Post-increment operator++(int) — saves copy, increments, returns saved copy	☐
18	Pre-decrement operator--() — subtracts 1, returns *this	☐
19	Post-decrement operator--(int) — saves copy, decrements, returns saved copy	☐

10.4 Comparison and Stream Operators

#	Item	Completed?
20	operator== — checks both isNegative and number	☐
21	operator!= — returns !(lhs == rhs)	☐
22	operator< — handles all four sign combinations	☐
23	operator<= — returns (lhs < rhs) (lhs == rhs)	☐
24	operator> — returns !(lhs <= rhs)	☐
25	operator>= — returns !(lhs < rhs)	☐
26	operator<< — prints '-' prefix when negative and non-zero	☐
27	operator>> — reads string token and constructs BigInt	☐
28	toString() — returns "-" + number or number	☐

11. Grading Criteria

Criterion	Weight	Description
Correctness of Output	40%	All 6 test groups produce the exact expected console output shown in Section 8
Edge Case Handling	25%	Division by zero exception, negative zero prevention, leading zero removal, zero operands all handled correctly
Code Quality	20%	Readable code, proper use of helpers, no code duplication across binary and compound-assignment operators
Sign Handling	10%	All sign combinations for +, -, *, /, % and comparisons are correct
Compilation	5%	Code compiles without warnings using: g++ -std=c++17 -Wall -Wextra

Appendix — Quick Reference Card

A concise summary for quick lookup during implementation.

Operator Summary

Operator	Signature	Returns
Default constructor	BigInt()	BigInt representing 0
Int constructor	BigInt(int64_t value)	BigInt from integer
String constructor	BigInt(const string& str)	BigInt from string
Negation	operator-()	Copy with flipped sign
Addition assign	operator+=(const BigInt& other)	*this (updated)
Subtraction assign	operator-=(const BigInt& other)	*this (updated)
Multiply assign	operator*=(const BigInt& other)	*this (updated)
Divide assign	operator/=(const BigInt& other)	*this (updated), throws on 0
Modulus assign	operator%=(const BigInt& other)	*this (updated), throws on 0
Pre-increment	operator++()	*this after increment
Post-increment	operator++(int)	copy before increment
Equality	operator==(lhs, rhs)	bool
Less-than	operator<(lhs, rhs)	bool
Output stream	operator<<(os, num)	os (for chaining)
Input stream	operator>>(is, num)	is (for chaining)
To string	toString() const	string representation

Sign Rule Quick Reference

Operation	Both positive	Both negative	Mixed signs
Addition	Add magnitudes, +	Add magnitudes, -	Subtract, sign of larger magnitude
Subtraction	Subtract, sign of larger magnitude	Subtract, sign of larger magnitude	Add magnitudes, sign of this
Multiply / Divide	Result is +	Result is +	Result is - (XOR rule)
Modulus	Result is +	Result is - (matches dividend)	Result sign matches dividend (this)

End of Document